

Ex No: 1:**Table Creation and Data Insertion:****SQL**

Create table Student(StudentID int PRIMARY KEY, Name varchar(30), Age int);

Create table Courses (CourseID int PRIMARY KEY, CourseName varchar(20));

Create table Enrollments (StudentID int REFERENCES students(student_id), CourseID int REFERENCES courses (course_id));

Insert into Student values(1,'Alice', 20);

Insert into Courses values(101, 'Database Management');

Insert into Enrollments values(1,101);

Simple Queries:

SELECT * FROM Students;

Output:

StudentID	Name	Age
-----------	------	-----

1	Alice	20
---	-------	----

2	Bob	22
---	-----	----

3	Charlie	21
---	---------	----

4	David	19
---	-------	----

SELECT Name, Age FROM Students WHERE Age>20;

Output:

StudentID	Name	Age
-----------	------	-----

2	Bob	22
---	-----	----

3	Charlie	21
---	---------	----

Nested Queries:

SELECT Name FROM Students WHERE StudentID IN (SELECT StudentID FROM Enrollments WHERE CourseID = (SELECT CourseID FROM Courses WHERE CourseName = 'Database Management'));

Output:

Name	StudentID
------	-----------

SELECT CourseID, CourseName FROM Courses WHERE CourseID IN (SELECT CourseID FROM Enrollments GROUP BY CourseID HAVING COUNT(*)>1);

Output:

CourseID	CourseName
----------	------------

101	Database Management
-----	---------------------

103	Web Development
-----	-----------------

Subqueries:

SELECT AVG(Age) AS AverageAge FROM Students;

Output:

AverageAge

```
SELECT Name, Age FROM Students WHERE Age > (SELECT AVG(Age) FROM Students);
```

Output:

Name Age
Bob 22
Charlie 21

Ex No: 2:**Table Creation and Data Insertion:****SQL**

Create table Student(StudentID int, Name varchar(30), Age int);

Create table Courses (CourseID int, CourseName varchar(20));

Create table Enrollments (EnrollmentID int, StudentID int, CourseID int, Grade varchar(5));

Insert into Student values(1,'Alice', 20);

Insert into Courses values (101,'Database Management');

Insert into Enrollments values(1,1,101,'A');

Inner Join:

```
SELECT Students.StudentID, Students.Name, Students.Age, Courses.CourseID,  
Courses.CourseName, Enrollments.Grade FROM Students INNER JOIN Enrollments ON  
Students.StudentID = Enrollments.StudentID INNER JOIN Courses ON  
Enrollments.CourseID = Courses.CourseID;
```

Output:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	1	Math	A
1	Alice	20	2	English	B
2	Bob	22	1	Math	A-
3	Charlie	21	3	History	B+
3	Charlie	21	2	English	A

Left Join:

```
SELECT Students.StudentID, Students.Name, Students.Age, Courses.CourseID,  
Courses.CourseName, Enrollments.Grade FROM Students LEFT JOIN Enrollments ON  
Students.StudentID = Enrollments.StudentID LEFT JOIN Courses ON Enrollments.CourseID  
= Courses.CourseID;
```

Output:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	1	Math	A
1	Alice	20	2	English	B
2	Bob	22	1	Math	A-
2	Bob	22	NULL	NULL	NULL
3	Charlie	21	3	History	B+
3	Charlie	21	2	English	A

Right Join:

```
SELECT Students.StudentID, Students.Name, Students.Age, Courses.CourseID,  
Courses.CourseName, Enrollments.Grade FROM Courses RIGHT JOIN Enrollments ON  
Courses.CourseID = Enrollments.CourseID RIGHT JOIN Students ON  
Enrollments.StudentID = Students.StudentID;
```

Output:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	1	Math	A
1	Alice	20	2	English	B
2	Bob	22	1	Math	A-
3	Charlie	21	3	History	B+
3	Charlie	21	2	English	A

Ex No: 3:**Table Creation and Data Insertion:**

SQL

```
CREATE TABLE students (student_id SERIAL PRIMARY KEY, student_name
VARCHAR(100), student_email VARCHAR(100));
INSERT INTO students (student_name, student_email) VALUES ('Alice Johnson',
'alice@example.com');
INSERT INTO students (student_name, student_email) VALUES ('Bob Smith',
'bob@example.com');
INSERT INTO students (student_name, student_email) VALUES ('Charlie Brown',
'charlie@example.com');
```

View Creation and DML Operations:

```
CREATE VIEW student_view AS SELECT student_id, student_name, student_email FROM
students;
INSERT INTO students (student_name, student_email) VALUES ('Diana Prince',
'diana@example.com');
UPDATE students SET student_email = 'new_bob@example.com' WHERE student_name =
'Bob Smith';
DELETE FROM students WHERE student_name = 'Charlie Brown';
SELECT * FROM student_view;
```

Output:

```
| student_id | student_name | student_email |
| 1 | Alice Johnson | alice@example.com |
| 2 | Bob Smith | new_bob@example.com |
| 4 | Diana Prince | diana@example.com |
```

Index Creation:

```
CREATE INDEX idx_student_email ON students (student_email);
```

Ex No: 4:**Implicit Cursors:**

```
DECLARE
total_rows number(2);
BEGIN
UPDATE customers
SET salary = salary + 500;
IF sql%notfound THEN dbms_output.put_line('no customers selected');
ELSIF sql%found THEN
total_rows := sql%rowcount;
dbms_output.put_line(total_rows || ' customers selected ');
END IF;
END;
/
```

Output:

6 customers selected
PL/SQL procedure successfully completed.

Explicit Cursors:

```
DECLARE
c_id customers.id%type;
c_name customer.name%type;
c_addr customers.address%type;
CURSOR c_customers is SELECT id, name, address FROM customers;
BEGIN
OPEN c_customers;
LOOP
FETCH c_customers into c_id, c_name, c_addr;
EXIT WHEN c_customers%notfound;
dbms_output.put_line(c_id || " " || c_name || " " || c_addr);
END LOOP;
CLOSE c_customers;
END;
/
```

Output:

ID	NAME	ADDRESS
1	Ramesh	Ahmadabad
2	Khilan	Delhi
3	Kaushik	Kota
4	Chaitali	Mumbai

Ex No: 5:**Procedure:**

```
set serveroutput on;
CREATE OR REPLACE PROCEDURE Sum(a IN number, b IN number) IS c number;
BEGIN
c := a+b;
dbms_output.put_line('Sum of two nos = ' || c);
END Sum;
/
```

Output:

```
Enter value for x: 10
Enter value for y: 20
Sum of two nos = 30
PL/SQL procedure successfully created.
```

Function:

```
set serveroutput on;
CREATE OR REPLACE FUNCTION Sum(a IN number, b IN number) RETURN Number
IS c number;
BEGIN
c := a+b;
RETURN c;
END;
/
```

Output:

```
Enter value for no1: 5
Enter value for no2: 5
Sum of two nos = 10
PL/SQL procedure successfully created.
```

Ex No: 6:

create or replace trigger up_classd before update on customer for each row

DECLARE

BEGIN

if updating then

DBMS_OUTPUT.PUT_LINE('new value is' || :new.stotal);

DBMS_OUTPUT.PUT_LINE('old value is'||:old.stotal);

end if;

END;

/

Output:

new value is;500

old value is;300

Trigger for Deletion:

create or replace trigger del_classb before delete on customer for each row

DECLARE

BEGIN

if deleting then

DBMS_OUTPUT.PUT_LINE('row deleted');

end if;

END;

/

Output:

row deleted

1 row deleted.

Trigger for Insertion:

create or replace trigger ins_classb before insert on classb for each row

DECLARE

InvTot exception;

BEGIN

if inserting then

if:new.stotal>1000 then

raise InvTot;

end if;

end if;

EXCEPTION

when InvTot then

raise application error(-20000, 'Total not valid');

END;

/

Output:

insert into classb values('6','jana', 'it', '20000', 'a')

ERROR at line 1:

ORA-20000: Total not valid

ORA-06512: at "SCOTT.INS_CLASSB", line 11

ORA-04088: error during execution of trigger 'SCOTT.INS_CLASSB'